

Implementação de Fila de Prioridade com Heap (Nó)

Interface Position

```
package Fila.Prioridade.Heap_No;

public interface Position {

    void insert(Object elemento, int key) throws InvalidPositionExceptionHeapNo;

    void swap(No node1, No node2) throws InvalidPositionExceptionHeapNo;

    int size();

    boolean isEmpty();

    void upheap(No node) throws InvalidPositionExceptionHeapNo;

    void downheap(No node) throws InvalidPositionExceptionHeapNo;

    No min() throws InvalidPositionExceptionHeapNo;

    void removeMin() throws InvalidPositionExceptionHeapNo;

    No getRoot() throws InvalidPositionExceptionHeapNo;

    No getLastNode() throws InvalidPositionExceptionHeapNo;

    No getInsertionPosition() throws InvalidPositionExceptionHeapNo;

    boolean hasLeft(No node) throws InvalidPositionExceptionHeapNo;

    boolean hasRight(No node) throws InvalidPositionExceptionHeapNo;

    boolean isRoot(No node) throws InvalidPositionExceptionHeapNo;

}
```

Classe No

```
package Fila.Prioridade.Heap_No;

public class No {
```

```

private int key;
private No parent, childrenEsq, childrenDir;
private Object elemento;

public No(Object elemento, int key){
    this.key = key;
    this.parent = null;
    this.childrenEsq = null;
    this.childrenDir = null;
    this.elemento = elemento;
}

public void setElement(Object elemento){
    this.elemento = elemento;
}

public void setKey(int key){
    this.key = key;
}

public void setParent(No parent){
    this.parent = parent;
}

public void setChildrenEsq(No childrenEsq){
    this.childrenEsq = childrenEsq;
}

public void setChildrenDir(No childrenDir){
    this.childrenDir = childrenDir;
}

public Object getElement(){
    return elemento;
}

public int getKey(){
    return key;
}

public No getParent(){
    return parent;
}

public No getChildrenEsq(){

```

```

        return childrenEsq;
    }

    public No getChildrenDir(){
        return childrenDir;
    }
}

```

Classe HeapNo

```

package Fila.Prioridade.Heap_No;

public class HeapNo implements Position {

    private No root;
    private int size;

    private No ultimo;
    private No posicaoInsercao;

    public HeapNo(){
        this.root = null;
        this.size = 0;
    }

    public void insert(Object elemento, int key){
        No novo = new No(elemento, key);

        if (isEmpty()){
            root = novo;
            size++;
            return;
        }

        No pai = getInsertionPosition();

        novo.setParent(pai);

        if (pai.getChildrenEsq() == null){
            pai.setChildrenEsq(novo);
        }
        else{
            pai.setChildrenDir(novo);
        }
    }
}

```

```

    }

    size++;
    upheap(novo);
}

public void swap(No node1, No node2) throws InvalidPositionExceptionHeapNo{
    if (node1 == null || node2 == null){
        throw new InvalidPositionExceptionHeapNo("Nó nulo");
    }

    Object elemento = node1.getElement();
    int key = node1.getKey();

    node1.setElement(node2.getElement());
    node1.setKey(node2.getKey());

    node2.setElement(elemento);
    node2.setKey(key);
}

public int size(){
    return size;
}

public boolean isEmpty(){
    return size == 0;
}

public void upheap(No node) throws InvalidPositionExceptionHeapNo{
    if (node == null){
        throw new InvalidPositionExceptionHeapNo("Nó nulo");
    }

    while (!isRoot(node)){
        if (node.getKey() < node.getParent().getKey()){
            swap(node, node.getParent());
            node = node.getParent();
        }
        else{
            break;
        }
    }
}
}

```

```

public void downheap(No node) throws InvalidPositionExceptionHeapNo{
    if (node == null){
        throw new InvalidPositionExceptionHeapNo("Nó nulo");
    }

    while (hasLeft(node)){
        No menorFilho = node.getChildrenEsq();

        if (hasRight(node) && node.getChildrenDir().getKey() < menorFilho.getKey()){
            menorFilho = node.getChildrenDir();
        }

        if (node.getKey() > menorFilho.getKey()){
            swap(node, menorFilho);
            node = menorFilho;
        }
        else{
            break;
        }
    }
}

public No min() throws InvalidPositionExceptionHeapNo{
    if (isEmpty()){
        throw new InvalidPositionExceptionHeapNo("Heap vazia");
    }

    return root;
}

public void removeMin() throws InvalidPositionExceptionHeapNo{
    if (isEmpty()){
        throw new InvalidPositionExceptionHeapNo("Heap vazia");
    }

    No ultimo = getLastNode();

    if (root == ultimo){
        root = null;
        size--;
        return;
    }

    root.setElement(ultimo.getElement());
    root.setKey(ultimo.getKey());
}

```

```

        No pai = ultimo.getParent();

        if (pai.getChildrenEsq() == ultimo){
            pai.setChildrenEsq(null);
        }
        else{
            pai.setChildrenDir(null);
        }

        size--;
        downheap(root);
    }

    public No getRoot() throws InvalidPositionExceptionHeapNo{
        if (isEmpty()){
            throw new InvalidPositionExceptionHeapNo("Heap vazia");
        }

        return root;
    }

    public No getLastNode() throws InvalidPositionExceptionHeapNo{
        if (isEmpty()){
            throw new InvalidPositionExceptionHeapNo("Heap vazia");
        }

        ultimo = null;
        getLastNodeRec(root);

        return ultimo;
    }

    private void getLastNodeRec(No node){
        if (node == null){
            return;
        }

        ultimo = node;

        getLastNodeRec(node.getChildrenEsq());
        getLastNodeRec(node.getChildrenDir());
    }

    public No getInsertionPosition() throws InvalidPositionExceptionHeapNo{
        if (root == null){

```

```

        throw new InvalidPositionExceptionHeapNo("Raiz nula");
    }

    posicaoInsercao = null;
    getInsertionPositionRec(root);

    return posicaoInsercao;
}

private void getInsertionPositionRec(No node){
    if (node == null || posicaoInsercao != null){
        return;
    }

    if (node.getChildrenEsq() == null || node.getChildrenDir() == null){
        posicaoInsercao = node;
        return;
    }

    getInsertionPositionRec(node.getChildrenEsq());
    getInsertionPositionRec(node.getChildrenDir());
}

public boolean hasLeft(No node) throws InvalidPositionExceptionHeapNo{
    if (node == null){
        throw new InvalidPositionExceptionHeapNo("Nó nulo");
    }

    return node.getChildrenEsq() != null;
}

public boolean hasRight(No node) throws InvalidPositionExceptionHeapNo{
    if (node == null){
        throw new InvalidPositionExceptionHeapNo("Nó nulo");
    }

    return node.getChildrenDir() != null;
}

public boolean isRoot(No node) throws InvalidPositionExceptionHeapNo{
    if (node == null){
        throw new InvalidPositionExceptionHeapNo("Nó nulo");
    }

    return node == root;
}

```

```
}  
}
```

Classe Exception

```
package Fila.Prioridade.Heap_No;  
  
public class InvalidPositionExceptionHeapNo extends Exception {  
    public InvalidPositionExceptionHeapNo(String err){  
        super(err);  
    }  
}
```